MODULE 0 · FOUNDATIONS

Container Runtime & the CRI

How the kubelet turns a Pod spec into a running container

WHY IT MATTERS

Kubernetes doesn't run containers — it delegates

The apiserver and scheduler decide **what** should run and **where**. Neither of them ever starts a container — that job belongs to the **kubelet**, the agent running on every node, which hands it off through a stable interface.

Every node-level problem — a stuck `ImagePullBackOff`, a node gone `NotReady`, a container hitting its memory limit — is this chain misbehaving somewhere.

BY THE END YOU CAN

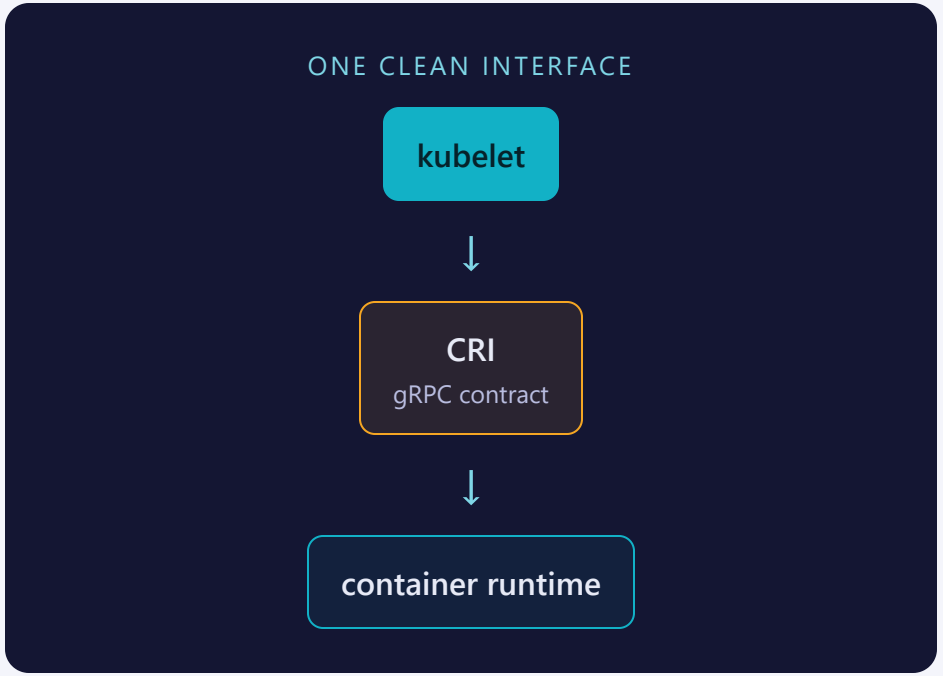
- Trace kubelet → CRI → containerd → runc
- Explain what the CRI is and why it exists
- Name what isolates a container at the kernel level
- Use `crictl` for node-local debugging

CONCEPT

The kubelet is a delegator, not a runtime

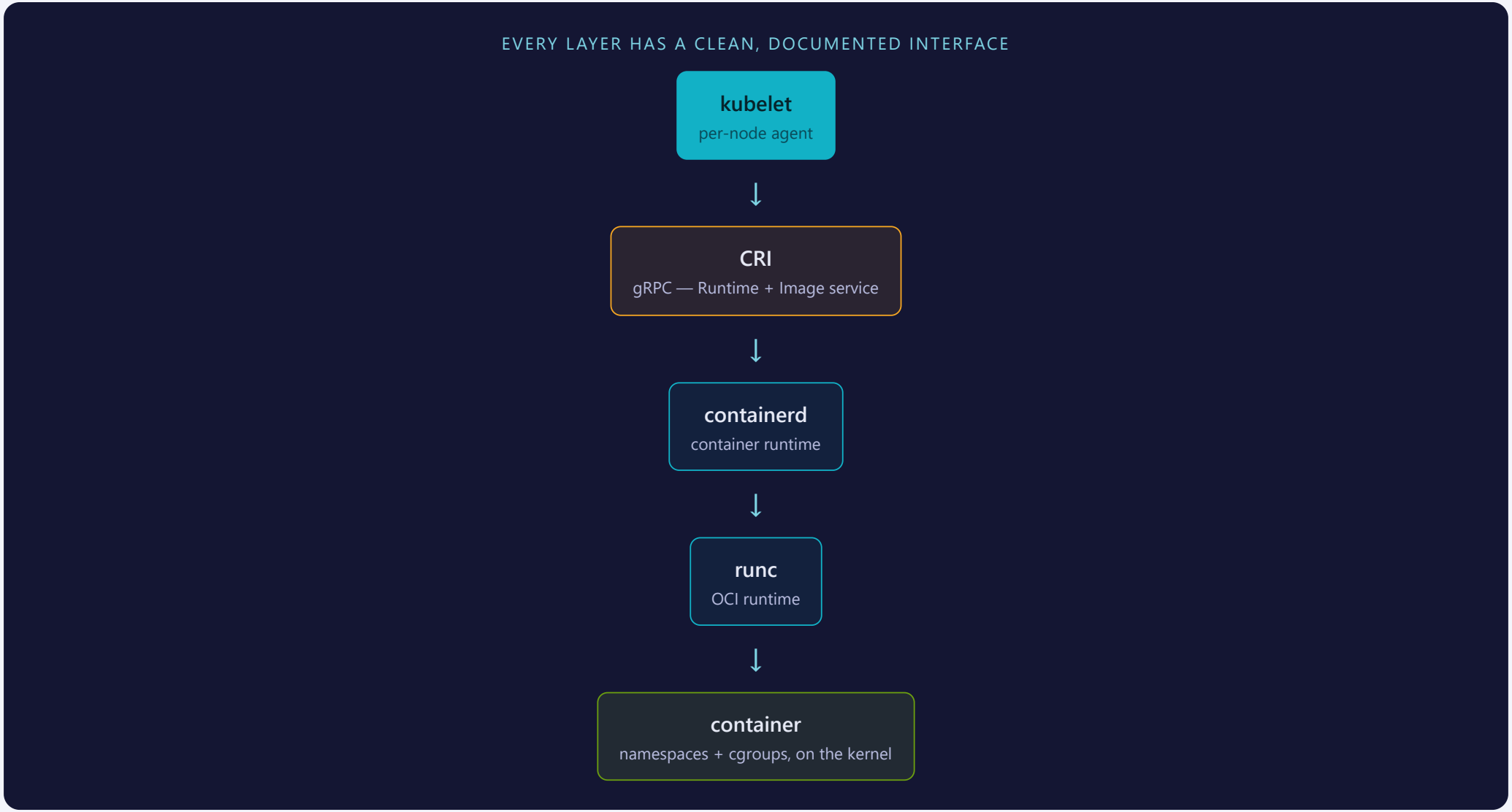
The kubelet's job is to keep the containers on its node matching the Pod specs the apiserver assigned it. It doesn't implement container execution itself — it calls out to a **container runtime** through a documented, versioned contract: the **CRI** (Container Runtime Interface).

That contract is what makes the runtime **swappable**: any runtime that implements CRI plugs in with no kubelet code changes.



THE CONTAINER STARTUP CHAIN

kubelet → CRI → containerd → runc → container



WHAT THE CRI ACTUALLY IS

One gRPC contract, two services

SERVICE	RESPONSIBLE FOR	EXAMPLE CALLS
RuntimeService	Pod sandboxes & containers	`RunPodSandbox`, `CreateContainer`, `StartContainer`
ImageService	Images cached on the node	`PullImage`, `ListImages`, `RemoveImage`

WHY IT EXISTS

Because the kubelet only ever speaks CRI, any compliant runtime plugs in unmodified — **containerd** and **CRI-O** are the common choices today.

WHAT ACTUALLY ISOLATES A CONTAINER

Namespaces + cgroups — that's the whole trick

NAMESPACES — WHAT IT CAN SEE	CGROUPS — WHAT IT CAN USE
PID — its own process tree	CPU — `requests` / `limits` → shares & throttling
Network — own interfaces, one IP	Memory — over the limit → `OOMKilled`
Mount — its own filesystem view	Both enforced by the kernel, set by the runtime

MENTAL MODEL

A container is just a Linux process with namespaces and cgroups applied to it — no separate kernel, no VM boundary.

CRICTL — NODE-LOCAL DEBUGGING

Talk to the runtime socket directly

	KUBECTL	CRICTL
Talks to	apiserver	runtime socket
Scope	cluster-wide	this node only
Needs	kubeconfig	runtime endpoint

```
$ crictl pods          # sandboxes
$ crictl ps            # containers
$ crictl images        # cached images
$ crictl pull nginx    # pull directly
```

Socket lives at `/run/containerd/containerd.sock` for a containerd node.

GOTCHAS — THE ONES THAT BITE

Where people trip

- **Blaming the apiserver for pull failures.** `ImagePullBackOff` is the runtime's job — check node registry access, credentials, image name.
- **Wrong or missing socket.** `crictl` needs the runtime endpoint; a missing socket means "can't connect" even when the cluster is healthy.
- **A dead runtime takes the node with it.** If containerd dies, the kubelet can't manage containers — Pods get evicted and rescheduled elsewhere.
- **Assuming images are cluster-wide.** They're cached per node — "works on one node, pulls on another" is usually just cache, not corruption.

RECAP

Container runtime & CRI in one breath

- kubelet delegates via CRI (gRPC) — never runs containers itself
- Chain: kubelet → CRI → containerd → runc → kernel
- RuntimeService (sandboxes/containers) + ImageService (images)
- Namespaces isolate what it sees; cgroups cap what it uses
- `crictl` talks to the runtime socket, node-local — `kubectl` talks to the apiserver, cluster-wide

Next: **Cluster Networking & the CNI**